

Types of Inheritance in C# with Examples

In this article, I am going to discuss **Types of Inheritance in C#** with Examples. Please read our previous article, where we discussed the basic concepts of [Inheritance in C#](#) Language with Examples.

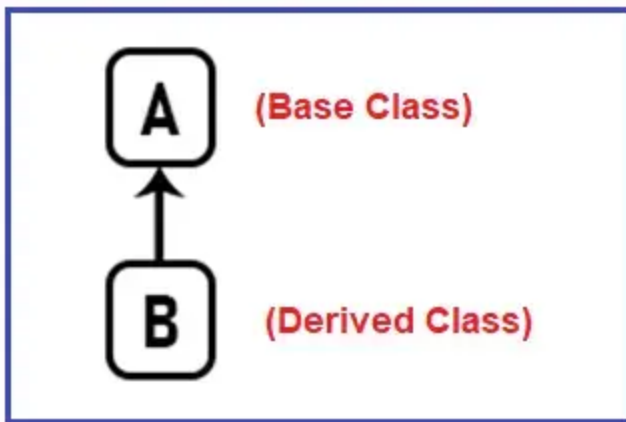
Types of Inheritance in C#

What these types of Inheritance will tell us is the number of parent classes a child class has or the number of child classes a parent class has. According to C++, why I am telling about C++ is because Object-Oriented Programming came into the picture from C++ only, there are five different types of Inheritances.

1. Single Inheritance
2. Multi-Level Inheritance
3. Hierarchical Inheritance
4. Hybrid Inheritance
5. Multiple Inheritance

Single Inheritance:

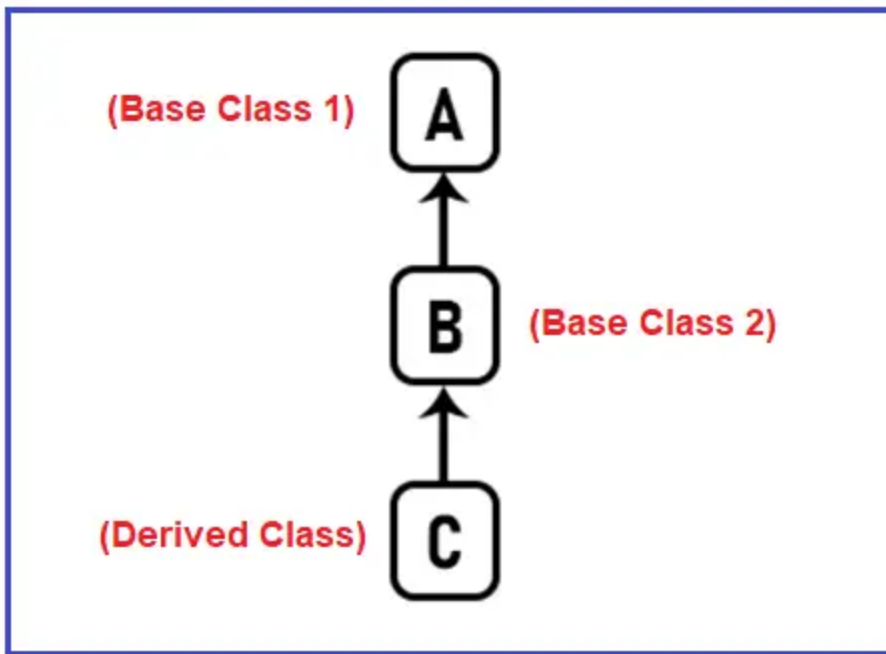
When a class is inherited from a single base class then the inheritance is called single inheritance. For a better understanding, please have a look at the below image.



As you can see in the above image, if we have a class called A that is the Parent class and another class called B that is the Child class, and class B is inheriting from class A. I.e. Class B has a single Parent class i.e. class A. This type of inheritance is called Single Inheritance.

Multilevel Inheritance:

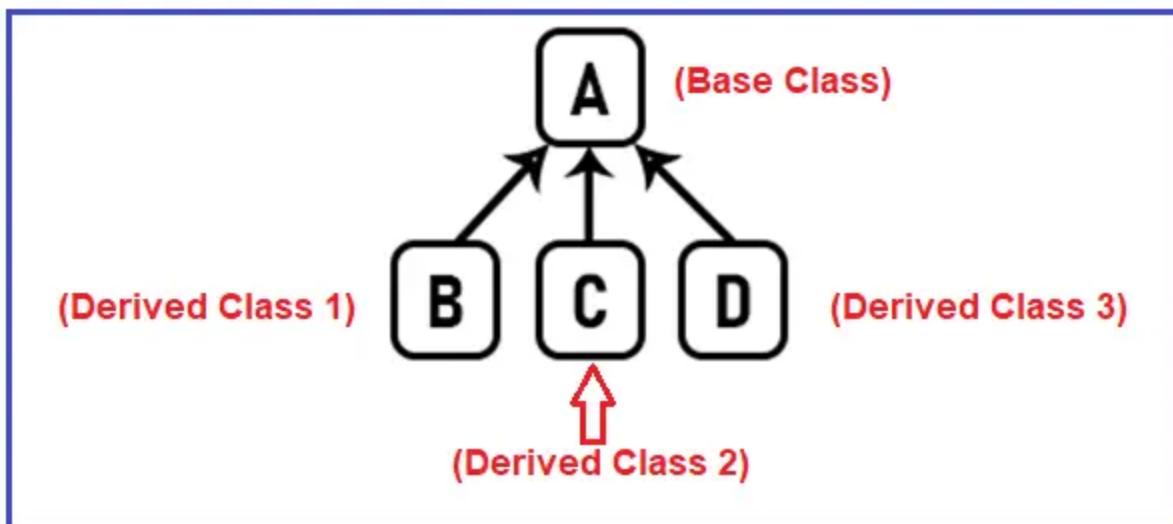
When a derived class is created from another derived class, then such a type of inheritance is called Multilevel Inheritance. For a better understanding, please have a look at the below image.



If there is a class called A and from class A, class B is inheriting and from class B, class C is inheriting, then such type of inheritance is called Multilevel Inheritance.

Hierarchical Inheritance:

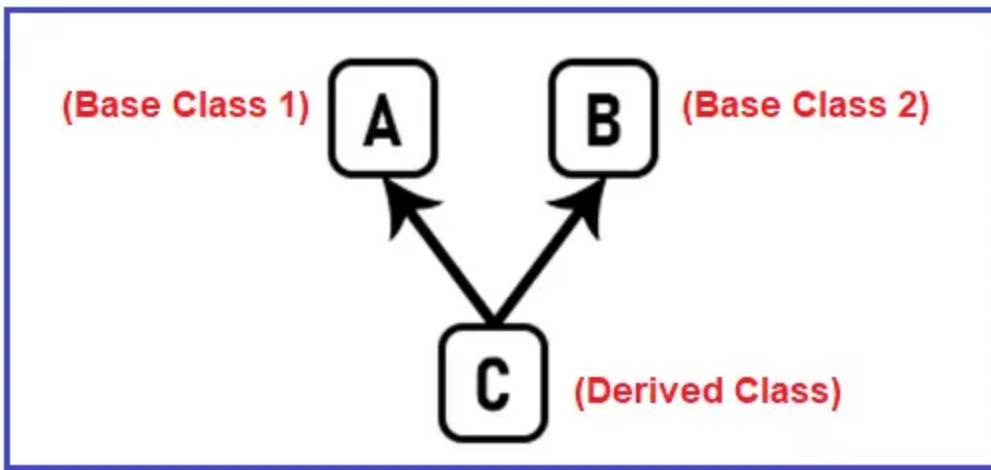
When more than one derived class is created from a single base class then it is called Hierarchical inheritance. For a better understanding, please have a look at the below image.



Now, if you have a class called A and from this class A, if more than one class inheriting i.e. class B is inheriting, class C is inheriting as well as class D is inheriting i.e. when more than one child class is inheriting from a Single Base Class, then such a type of inheritance is called Hierarchical Inheritance.

Multiple Inheritance:

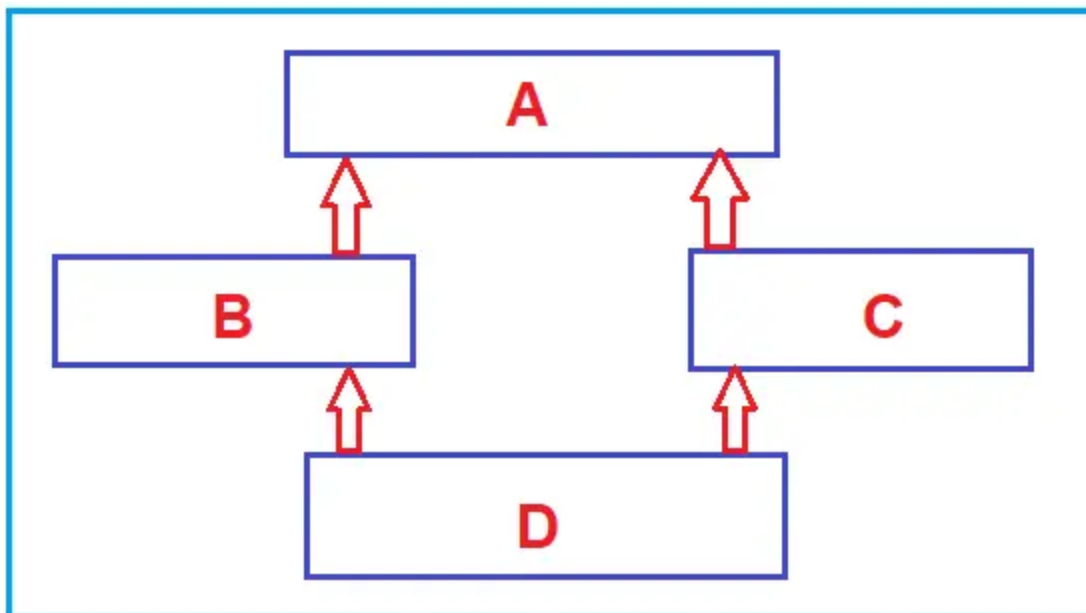
When a derived class is created from more than one base class then such type of inheritance is called multiple inheritances. For a better understanding, please have a look at the below image.



If there are classes A and B and from both of them class C is inheriting, then such type of inheritance is called Multiple Inheritance. So, when one class is having multiple parent classes then such type of inheritance is called Multiple Inheritance.

Hybrid Inheritance:

Hybrid Inheritance is the inheritance that is the combination of any Single, Hierarchical, and Multilevel inheritances. For a better understanding, please have a look at the below image.



There are two subclasses i.e. B and C which are inheriting from class A (this is Hierarchical inheritance). Then from B and C, there is one more class that is D inherits from B and C. Now, this is a combination of hierarchical inheritance from the top and multiple inheritances (D is inheriting from B and C) from the bottom. Further, from A to B and from B to C i.e. Multi-level Inheritance. So, if you have this type of inheritance then the features of base class A will be appearing in class D via class B and class C. This type of Inheritance is called Hybrid Inheritance.

The above classification is based on C++.

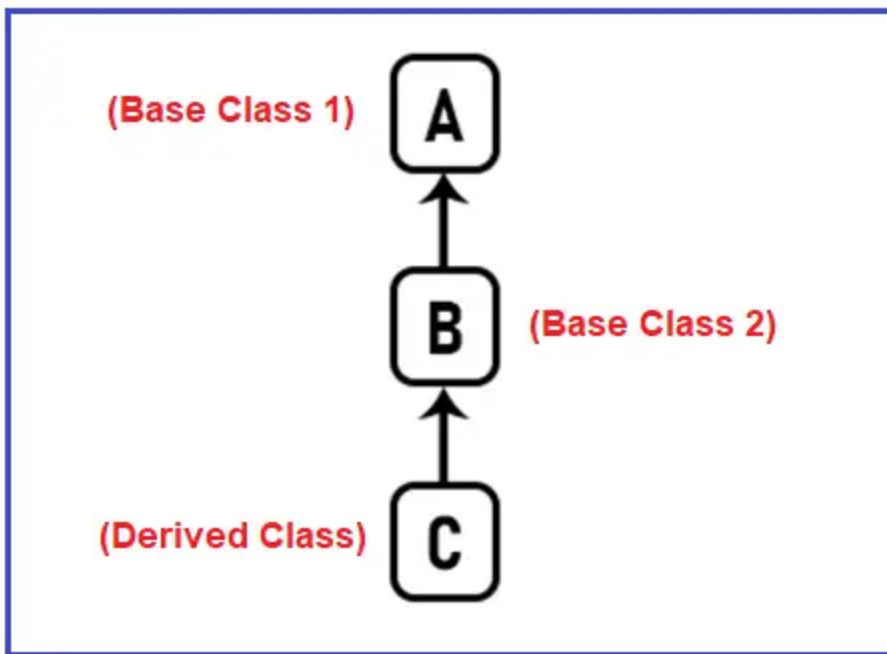
Types of Inheritances in C#:

If you look at Single, Multilevel, and Hierarchical inheritances, they all look very similar. At any point in time, they are having a single immediate parent class. But, if you look at Multiple and Hybrid, they are having more than one immediate parent class for a child class. So, we can broadly classify the above five categories of inheritances into two types based on immediate parent class as follows:

1. **Single Inheritance (Single, Multilevel, and Hierarchical)**
2. **Multiple Inheritance (Multiple and Hybrid)**

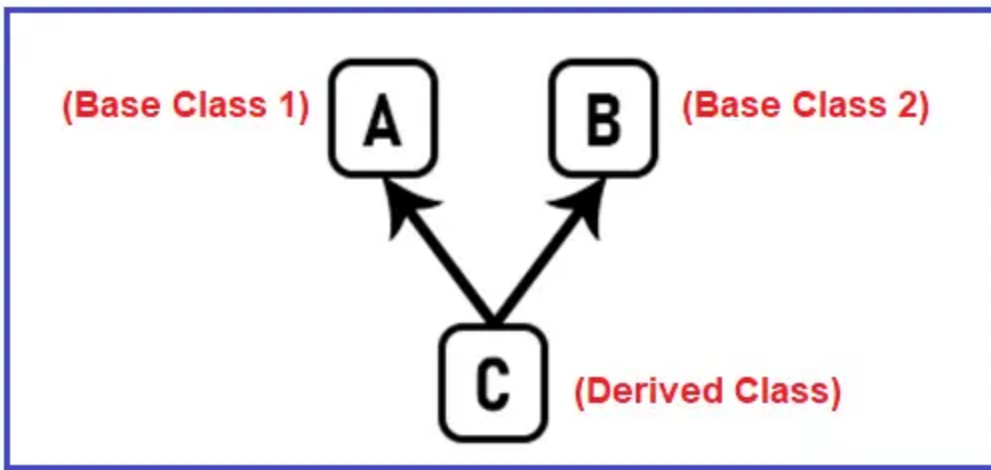
Single Inheritance in C#:

If at all a class has 1 immediate parent class to it, we call it Single Inheritance in C#. For a better understanding, please have a look at the below diagram. See, how many immediate parent class C has? 1 i.e. B, and how many immediate parent class B has? 1 i.e. A. So, for class C, the immediate Parent is class B and for class B, the immediate Parent is class A.



Multiple Inheritance in C#:

If a class has more than 1 immediate parent class to it, then we call it Multiple Inheritance in C#. For a better understanding, please have a look at the below diagram. See, class C has more than one immediate Parent class i.e. A and B and hence it is Multiple Inheritance.



So, the point that you need to remember is how many immediate Parent classes a child class has. If one immediate Parent class, call it Single Inheritance, and if more than one immediate Parent class, call it is multiple inheritance. So, there should not be any confusion between 5 different types of inheritances, simply we have two types of inheritances.

Example to Understand Single Inheritance in C#:

```
using System;
namespace InheritanceDemo
{
    public class Program
    {
        static void Main()
        {
            // Creating object of Child class and
            // invoke the methods of Parent and Child classes
            Cuboid obj = new Cuboid(2, 4, 6);
            Console.WriteLine($"Volume is : {obj.Volume()}");
            Console.WriteLine($"Area is : {obj.Area()}");
            Console.WriteLine($"Perimeter is : {obj.Perimeter()}");
            Console.ReadKey();
        }
    }
    //Parent class
    public class Rectangle
    {
        public int length;
        public int breadth;
        public int Area()
        {
            return length * breadth;
        }
        public int Perimeter()
        {
            return 2 * (length + breadth);
        }
    }
}
```

```

//Child Class
class Cuboid : Rectangle
{
    public int height;
    public Cuboid(int l, int b, int h)
    {
        length = l;
        breadth = b;
        height = h;
    }
    public int Volume()
    {
        return length * breadth * height;
    }
}
}

```

Output:

```

Volume is : 48
Area is : 8
Perimeter is : 12

```

Example to Understand Multiple Inheritance in C#:

```

using System;
namespace InheritanceDemo
{
    public class Program
    {
        static void Main()
        {
            // Creating object of Child class and
            // invoke the methods of Parent classes and Child class
            SmartPhone obj = new SmartPhone(); ;
            obj.GetPhoneModel();
            obj.GetCameraDetails();
            obj.GetDetails();
            Console.ReadKey();
        }
    }
}
//Parent Class 1
class Phone
{
    public void GetPhoneModel()
    {
        Console.WriteLine("Redmi Note 5 Pro");
    }
}

```

```

}
//Parent class2
class Camera
{
    public void GetCameraDetails()
    {
        Console.WriteLine("24 Mega Pixel Camera");
    }
}
//Child Class derived from more than one Parent class
class SmartPhone : Phone, Camera
{
    public void GetDetails()
    {
        Console.WriteLine("Its a Redmi Smart Phone");
    }
}
}

```

Output: Compile Time Error

Note: Handling the complexity caused due to multiple inheritances is very complex. Hence it was not supported in dot net with class and it can be done with interfaces. So, in our [Multiple Inheritance](#) articles, we will discuss this concept in detail.

Classification of Inheritance in C#

C#.NET classified the inheritance into two categories, such as

1. **Implementation inheritance:** Whenever a class is derived from another class then it is known as implementation inheritance.
2. **Interface inheritance:** Whenever a class is derived from an interface then it is known as interface inheritance.

In the next article, I am going to discuss [How to use inheritance in Realtime Application Development](#) with Examples using C# Language. Here, in this article, I try to explain the **Types of Inheritance in C#** with Examples and I hope you enjoy this Types of Inheritance in C# article.